BIZEV-03: Funnel Analysis

> **Series:** BIZEV — Business Events & Funnel Analysis | **Notebook:** 3 of 7 | **Created:** March 2026 | **Last Updated:** 10/25/2026

Overview

Conversion funnel analysis is one of the most powerful applications of business events. By tracking users through a sequence of steps — browse, add to cart, checkout, purchase — you can identify where customers drop off, measure conversion rates, and quantify the business impact of friction points. This notebook demonstrates how to build multi-step funnels from business events using DQL, calculate step-by-step conversion rates, and analyze the time users spend between funnel stages.

Table of Contents

1. Funnel Analysis Concepts
2. Building a Basic Funnel
3. Step-by-Step Conversion Rates
4. Identifying Drop-Off Points
5. Time Between Steps
6. Funnel Segmentation
7. Funnel Trends Over Time
8. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
Permissions	`storage:bizevents:read`
Data	Business events with sequential event types (e.g., browse, cart, checkout, purchase)
Knowledge	BIZEV-01 fundamentals, DQL `summarize` and `countIf`

1. Funnel Analysis Concepts

> **Restrict Business Flow to specific buckets (SaaS 1.346 — staged rollout from 08/25/2026).** Verbatim: *"in the Business Flow edit mode, it's possible to restrict business flow analysis to one or more specific business event buckets (up to 10 selected buckets), so all DQL queries will scan a smaller amount of bytes, increasing performance in the queries and reducing DPS license consumption on every run."* This matters more than a performance tweak: a Business Flow re-runs its DQL on **every** view, so an unrestricted flow scanning all bizevent buckets bills that scan repeatedly. If your bizevents are already partitioned into buckets — the ORGNZ pattern — selecting the relevant ones (max 10) cuts both latency and per-run DPS. The funnel DQL below is unchanged; the restriction applies at the Business Flow configuration level, not in the query text.
>
> Source: What's new in Dynatrace SaaS 1.346 (DT docs)

A **conversion funnel** models a sequence of steps that users take to complete a desired outcome. Each step represents a business event type.

Example: E-Commerce Funnel

Step	Event Type	Description
1	`com.myapp.product.viewed`	User views a product page

Step	Event Type	Description
2	`com.myapp.cart.updated`	User adds item to cart
3	`com.myapp.checkout.started`	User begins checkout
4	`com.myapp.payment.processed`	Payment completed
5	`com.myapp.order.completed`	Order confirmed

Key Metrics

Metric	Formula	Meaning
Step conversion rate	Step N count / Step N-1 count	% proceeding to next step
Overall conversion rate	Last step / First step	End-to-end conversion
Drop-off rate	1 - step conversion rate	% leaving at each step
Time to convert	Avg time from first to last step	Speed of conversion

2. Building a Basic Funnel

The simplest funnel counts **events** at each step with `countIf`, in a single query. Event counts are a quick health signal but not a conversion rate: a user who updates the cart three times is three `cart.updated` events, so a step can exceed the one before it. For user-level conversion, count distinct users or sessions per step — the second query below.

> **Note:** Adapt the `event.type` values below to match your actual business event names. The examples use a generic e-commerce pattern.

```
// Basic funnel — count events at each step over the last 24 hours
fetch bizevents, from:-24h
| summarize {step1_view = countIf(event.type == "com.myapp.product.viewed"),
           step2_cart = countIf(event.type == "com.myapp.cart.updated"),
           step3_checkout = countIf(event.type ==
"com.myapp.checkout.started"),
           step4_payment = countIf(event.type ==
"com.myapp.payment.processed"),
           step5_complete = countIf(event.type ==
"com.myapp.order.completed")}
```

```
// Funnel with distinct user counts (more accurate for conversion)
// Uses countDistinctApprox for performance on large datasets
fetch bizevents, from:-24h
| filter isNotNull(user_id)
| summarize {step1_users = countDistinctApprox(if(event.type ==
"com.myapp.product.viewed", then: user_id)),
           step2_users = countDistinctApprox(if(event.type ==
"com.myapp.cart.updated", then: user_id)),
           step3_users = countDistinctApprox(if(event.type ==
"com.myapp.checkout.started", then: user_id)),
           step4_users = countDistinctApprox(if(event.type ==
"com.myapp.payment.processed", then: user_id)),
           step5_users = countDistinctApprox(if(event.type ==
"com.myapp.order.completed", then: user_id))}
```

3. Step-by-Step Conversion Rates

Conversion rates reveal the percentage of users who proceed from one step to the next. This is the core metric for funnel optimization. The query below divides **event** counts for brevity — swap in the distinct-user counts from § 2 when a step can repeat per user, or a rate above 100% will appear.

```
// Calculate conversion rates between each funnel step
fetch bizevents, from:-24h
| summarize {step1 = countIf(event.type == "com.myapp.product.viewed"),
           step2 = countIf(event.type == "com.myapp.cart.updated"),
           step3 = countIf(event.type == "com.myapp.checkout.started"),
           step4 = countIf(event.type == "com.myapp.payment.processed"),
           step5 = countIf(event.type == "com.myapp.order.completed")}
| fieldsAdd view_to_cart = round(toDouble(step2) / toDouble(step1) * 100,
decimals: 1),
           cart_to_checkout = round(toDouble(step3) / toDouble(step2) * 100,
decimals: 1),
           checkout_to_payment = round(toDouble(step4) / toDouble(step3) *
100, decimals: 1),
           payment_to_complete = round(toDouble(step5) / toDouble(step4) *
100, decimals: 1),
           overall_conversion = round(toDouble(step5) / toDouble(step1) * 100,
decimals: 2)
```

4. Identifying Drop-Off Points

Drop-off analysis highlights where users abandon the process. The step with the highest drop-off rate is your biggest optimization opportunity.

```
// Visualize funnel steps as a bar chart to see drop-off
// Each row represents one funnel step
fetch bizevents, from:-24h
| filter in(event.type, {"com.myapp.product.viewed", "com.myapp.cart.updated",
"com.myapp.checkout.started", "com.myapp.payment.processed",
"com.myapp.order.completed"})
| summarize event_count = count(), by:{event.type}
| sort event_count desc
```

```
// Identify checkout abandonment — users who started checkout but did not pay
// Requires a user_id or session_id field in your business events
fetch bizevents, from:-24h
| filter in(event.type, {"com.myapp.checkout.started",
"com.myapp.payment.processed"})
| filter isNotNull(user_id)
| summarize {started_checkout = countIf(event.type ==
"com.myapp.checkout.started"),
           completed_payment = countIf(event.type ==
"com.myapp.payment.processed")}, by:{user_id}
| filter started_checkout &gt; 0 and completed_payment == 0
| summarize abandoned_users = count()
```

5. Time Between Steps

Understanding how long users take between funnel steps reveals friction points. A long delay between "add to cart" and "checkout" may indicate a confusing checkout flow or price comparison behavior.

```
// Average time between cart and checkout (per user)
// Requires a user_id or session identifier
fetch bizevents, from:-24h
| filter in(event.type, {"com.myapp.cart.updated",
"com.myapp.checkout.started"})
| filter isNotNull(user_id)
| summarize {cart_time = min(if(event.type == "com.myapp.cart.updated", then:
timestamp)),
           checkout_time = min(if(event.type == "com.myapp.checkout.started",
then: timestamp))}, by:{user_id}
| filter isNotNull(cart_time) and isNotNull(checkout_time)
| fieldsAdd time_to_checkout_sec =
(toDouble(unixMillisFromTimestamp(checkout_time)) -
toDouble(unixMillisFromTimestamp(cart_time))) / 1000.0
| filter time_to_checkout_sec &gt; 0
| summarize {avg_seconds = avg(time_to_checkout_sec),
           median_seconds = median(time_to_checkout_sec),
           p95_seconds = percentile(time_to_checkout_sec, 95)}
```

6. Funnel Segmentation

Segmenting funnels by dimensions — device type, geographic region, customer tier — reveals which segments convert better and which need attention.

```
// Funnel conversion rates segmented by event provider
// Useful for comparing web vs mobile vs API channels
fetch bizevents, from:-24h
| summarize {step1 = countIf(event.type == "com.myapp.product.viewed"),
           step5 = countIf(event.type == "com.myapp.order.completed")}, by:
{event.provider}
| filter step1 &gt; 0
| fieldsAdd conversion_pct = round(toDouble(step5) / toDouble(step1) * 100,
decimals: 1)
| sort conversion_pct desc
```

```
// Funnel by business line — group on a payload field such as product_category
fetch bizevents, from:-24h
| summarize {views = countIf(event.type == "com.myapp.product.viewed"),
           purchases = countIf(event.type == "com.myapp.order.completed")},
by:{product_category}
| filter views &gt; 0
| fieldsAdd conversion_pct = round(toDouble(purchases) / toDouble(views) *
100, decimals: 1)
| sort conversion_pct desc
```

7. Funnel Trends Over Time

Track how conversion rates change over time to measure the impact of feature releases, marketing campaigns, or incidents.

```
// Funnel step volumes over time
fetch bizevents, from:-7d
| filter in(event.type, {"com.myapp.product.viewed", "com.myapp.cart.updated",
"com.myapp.checkout.started", "com.myapp.order.completed"})
| makeTimeseries event_count = count(), by:{event.type}, interval:6h
```

```
// Daily overall conversion rate trend over the past week
fetch bizevents, from:-7d
| fieldsAdd day = bin(timestamp, 24h)
| summarize {views = countIf(event.type == "com.myapp.product.viewed"),
           orders = countIf(event.type == "com.myapp.order.completed")}, by:
{day}
| filter views &gt; 0
| fieldsAdd conversion_pct = round(toDouble(orders) / toDouble(views) * 100,
decimals: 2)
| sort day asc
```

8. Summary and Next Steps

In this notebook, you learned:

- **Funnel concepts** — Steps, conversion rates, drop-off rates
- **Building funnels with DQL** — Using `countIf` to count each step in a single query
- **Conversion rate calculation** — Step-by-step and overall rates
- **Drop-off analysis** — Identifying abandoned sessions
- **Time between steps** — Measuring friction via inter-step timing
- **Segmentation** — Comparing funnels by provider, category, or other dimensions
- **Trend analysis** — Tracking conversion rates over time

Next Steps

- **BIZEV-04: Revenue Impact** — Connect funnel drop-offs to revenue loss during incidents
- **BIZEV-05: KPIs and Metrics** — Define business KPIs from event data

References

- Dynatrace Business Analytics
- DQL structuring commands (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace

documentation.